

Lumière Cinémas

Application Web de Réservation de Cinéma
Rapport Technique Complet

Spécifications Techniques

Plateforme	Web	Next.js 15 (App Router)
Langage	TypeScript	+ React 19
Base de données	Supabase	(PostgreSQL)
Paielement	Stripe	Checkout
Date	22 mai 2026	

Informations Académiques

Étudiants	Wassim Lazim Marwan Saadouni Kamal Chaafie
Encadrante	NAJI Zineb
Type	Projet de Fin d'Études
Établissement	École Marocaine des Sciences de l'Ingénieur

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Dédicaces

À ma famille qui m'a soutenu tout au long de ce parcours,

À mes amis qui ont cru en ce projet,

*À mon encadrante NAJI Zineb pour ses conseils précieux et
son accompagnement,*

*À tous ceux qui ont contribué de près ou de loin à la réussite
de cette application.*

Wassim Lazim

Remerciements

Je tiens à exprimer mes plus sincères remerciements à :

- **NAJI Zineb**, mon encadrante, pour son orientation précieuse, ses conseils avisés et sa disponibilité constante tout au long de ce projet de fin d'études.
- **Les membres du jury** pour le temps consacré à l'évaluation de ce rapport et de l'application.
- **L'ensemble du corps enseignant** de l'École Marocaine des Sciences de l'Ingénieur pour la qualité de la formation dispensée et les connaissances transmises tout au long de ces années.
- **Ma famille et mes proches** pour leur soutien moral indéfectible et leurs encouragements continus tout au long de cette formation.
- **Mes camarades de promotion** pour l'entraide, les échanges et la bonne ambiance qui ont rendu ce parcours enrichissant.

Résumé

Lumière Cinémas est une application web complète de réservation de cinéma développée avec Next.js 15 (App Router) et React 19. Elle permet aux utilisateurs de parcourir un catalogue de films, de consulter les séances disponibles, de sélectionner leurs places sur un plan interactif de salle, d'ajouter des snacks, et de procéder au paiement en ligne via Stripe.

L'application intègre :

- Un back-end cloud **Supabase** (PostgreSQL + Auth) pour l'authentification, la gestion des données et la Row-Level Security
- Un rendu **Server-Side Rendering** (SSR) avec React Server Components et Server Actions Next.js pour la sécurité des opérations critiques
- Un tunnel de paiement sécurisé via **Stripe Checkout**, avec écriture de la réservation déclenchée par le webhook de confirmation de paiement
- Un composant **SeatPicker** interactif calculant les disponibilités en temps réel depuis Supabase, prévenant toute double-réservation
- Un stepper de navigation en 6 étapes : Films → Séance → Places → Snacks → Paiement → Billet
- Un **Dashboard administrateur** complet protégé par middleware Next.js et RLS Supabase (gestion CRUD films, séances, snacks, réservations, utilisateurs)
- Un état de réservation intermédiaire (*Draft*) persisté en cookie HTTP pour traverser le tunnel sans perte de données

Ce rapport détaille l'architecture SSR + Server Actions, les choix techniques fondamentaux, le modèle de données Supabase, les fonctionnalités principales, l'implémentation technique complète avec extraits de code réels, les diagrammes UML, et les perspectives d'évolution du projet.

Mots-clés : Application Web, Next.js 15, React 19, TypeScript, Supabase, PostgreSQL, Stripe, SSR, Server Actions, Tailwind CSS, Réservation, Cinéma, SeatPicker

Table des matières

Résumé	5
Liste des Abréviations	10
1 Introduction et Contexte Général du Projet	11
1.1 Présentation du domaine	11
1.2 Présentation du projet	11
1.3 Motivation et Objectifs	11
1.4 Problématique	12
2 Planification du Projet	13
2.1 Cycle de Vie du Projet	13
2.2 Méthodologie de Développement	14
2.3 Découpage en Phases et Jalons	14
2.4 Diagramme de Gantt	15
3 Analyse des Besoins	16
3.1 Acteurs du Système	16
3.2 Besoins Fonctionnels	16
3.3 Besoins Non-Fonctionnels	17
3.4 Cas d'Usage	17
4 Architecture et Conception	19
4.1 Architecture Générale	19
4.2 Structure des Routes Next.js	20
4.3 Modèle de Données	20
4.4 Gestion de l'État Le Draft Cookie	22
4.5 Diagramme d'Activité Tunnel de Réservation	22
5 Stack Technologique	24
5.1 Tableau des Technologies	24
5.2 Justification des Choix	24
6 Fonctionnalités et Interface Utilisateur	26
6.1 Page d'Accueil Catalogue et HeroCarousel	26
6.2 Fiche Film et Bande-Annonce	27
6.3 Stepper Navigation Guidée	28
6.4 Sélection des Séances	28
6.5 Sélection des Places SeatPicker	28

6.6	Sélection des Snacks	30
6.7	Paiement Stripe Checkout	30
6.8	Confirmation et Billet	31
6.9	Espace Utilisateur	32
6.9.1	Authentification	32
6.9.2	Historique des Réservations	32
6.10	Dashboard Administrateur	33
6.10.1	Protection et Accès	33
6.10.2	Vue d'Ensemble	33
6.10.3	Gestion des Films	33
6.10.4	Gestion des Séances	34
6.10.5	Gestion des Réservations et Utilisateurs	34
7	Implémentation Technique	35
7.1	Génération du Plan de Salle	35
7.2	Création de la Session Stripe	35
7.3	Protection du Dashboard Admin	36
7.4	Diagrammes de Séquence	37
7.4.1	Flux de Réservation Complet	37
7.4.2	Flux de Paiement Stripe	39
8	Tests et Validation	40
8.1	Tests Fonctionnels Réalisés	40
8.2	Tests à Implémenter	40
9	Mise en Route et Configuration	41
9.1	Variables d'Environnement	41
9.2	Lancement en Développement	41
9.3	Base de Données Supabase	41
10	Perspectives Futures	43
10.1	Court Terme	43
10.2	Moyen Terme	43
10.3	Long Terme	43
	Conclusion Générale	44
A	Structure Complète du Projet	45
B	Ressources et Références	47

Table des figures

2.1	Diagramme de Gantt Planification du projet Lumière Cinémas (01/05 10/06/2026)	15
3.1	Diagramme des cas d'usage Lumière Cinémas	18
4.1	Architecture technique Lumière Cinémas	20
4.2	Diagramme de classes Modèle de données Supabase	21
4.3	Diagramme d'activité Tunnel de réservation	23
6.1	Page d'accueil HeroCarousel et grille de films	26
6.2	Grille du catalogue de films	27
6.3	Fiche détaillée d'un film avec bande-annonce et séances	27
6.4	Stepper indicateur de progression du tunnel	28
6.5	Page de sélection de séance avec disponibilités	28
6.6	Composant SeatPicker Plan interactif de la salle	29
6.7	SeatPicker avec sièges sélectionnés et panneau récapitulatif	29
6.8	Page de sélection des snacks	30
6.9	Page de paiement récapitulatif et options de règlement	31
6.10	Billet de confirmation avec référence de réservation	31
6.11	Pages d'authentification	32
6.12	Historique des réservations de l'utilisateur	32
6.13	Dashboard administrateur Vue d'ensemble et KPIs	33
6.14	Dashboard Gestion des films (CRUD)	34
6.15	Dashboard Gestion des séances	34
6.16	Dashboard Réservations et utilisateurs	34
7.1	Diagramme de séquence Tunnel de réservation	38
7.2	Diagramme de séquence Paiement Stripe	39

Liste des tableaux

2.1 Cycle de vie du projet Lumière Cinémas	13
--	----

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
App Router	Système de routage basé sur le système de fichiers (Next.js 13+)
Auth	Authentication (Authentification)
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
Draft	Brouillon de réservation, persisté en cookie HTTP
ENV	Environment Variables (Variables d'environnement)
HTTP/HTTPS	HyperText Transfer Protocol / Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAD	Dirham marocain (devise de paiement)
Middleware	Couche d'interception des requêtes Next.js
ORM	Object-Relational Mapping
PostgreSQL	Système de gestion de base de données relationnelle open-source
REST	Representational State Transfer
RLS	Row Level Security (politique d'accès par ligne Supabase)
RSC	React Server Components
SDK	Software Development Kit
SSR	Server-Side Rendering
Supabase	Backend-as-a-Service Open Source (PostgreSQL + Auth + Realtime)
TypeScript	Sur-ensemble typé de JavaScript développé par Microsoft
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

Chapitre 1

Introduction et Contexte Général du Projet

1.1 Présentation du domaine

L'industrie cinématographique s'est profondément numérisée au cours de la dernière décennie. La réservation en ligne est désormais un standard incontournable pour tout complexe cinématographique, exigeant des plateformes web modernes, réactives et sécurisées couvrant l'intégralité du parcours client : découverte du catalogue, sélection de la séance, choix des places, et paiement.

Au Maroc, ce secteur connaît une forte croissance avec l'ouverture de nouveaux multiplexes, rendant la nécessité d'une solution digitale performante encore plus pressante.

1.2 Présentation du projet

Lumière Cinémas est une application web full-stack de réservation de cinéma construite avec Next.js 15 et son App Router. Elle offre :

- Une **vitrine publique** : catalogue de films en carousel, fiches détaillées avec bande-annonce YouTube intégrée, liste des séances avec disponibilités en temps réel
- Un **tunnel de réservation** en 6 étapes guidées (stepper) : Films → Séance → Places → Snacks → Paiement → Billet
- Un **espace personnel** : inscription, connexion, historique des réservations
- Un **back-office administrateur** complet pour la gestion du cinéma

1.3 Motivation et Objectifs

Le projet répond à plusieurs objectifs pédagogiques et techniques :

- Maîtriser les technologies web modernes de pointe (Next.js 15 App Router, React 19, TypeScript)
- Implémenter une architecture SSR sécurisée avec Server Actions (zéro exposition des clés côté client)
- Intégrer des services tiers de production (Supabase Auth, Stripe Checkout)

- Concevoir un composant interactif complexe (plan de salle 2D avec catégories de sièges)
- Gérer un état de session multi-pages sans framework de state management externe
- Produire une interface utilisateur soignée avec une identité visuelle cohérente

1.4 Problématique

L'enjeu central était de concevoir une plateforme unifiée gérant simultanément la vitrine client et le back-office administrateur, avec une sécurité totale sur les opérations sensibles (paiement, gestion des réservations), sans exposer les clés d'API côté navigateur, et en garantissant la cohérence des données face aux réservations concurrentes (deux utilisateurs sur les mêmes sièges).

Chapitre 2

Planification du Projet

2.1 Cycle de Vie du Projet

Le projet Lumière Cinémas s'appuie sur un cycle de vie **itératif et incrémental**, organisé en cinq phases successives et interdépendantes. Contrairement au modèle en cascade classique, ce cycle autorise des retours entre phases afin d'intégrer les ajustements au fur et à mesure du développement.

N°	Phase	Activités principales	Livrables
1	Analyse des besoins	Identification des acteurs, recueil des besoins fonctionnels et non-fonctionnels, définition des cas d'usage	Cahier des charges, diagramme de cas d'usage
2	Conception	Architecture technique, modélisation UML (classes, séquences, activité), conception du schéma de données Supabase	Diagrammes UML, schéma BDD
3	Développement	Implémentation incrémentale : authentification, vitrine publique, tunnel de réservation, intégration Stripe, dashboard admin	Code source TypeScript, modules fonctionnels
4	Tests & Validation	Vérification fonctionnelle de chaque module, tests de sécurité (accès admin, protection des clés API, cohérence des réservations)	Tableau de tests, corrections
5	Déploiement	Mise en production sur Vercel, configuration des variables d'environnement, documentation technique finale	Application déployée, rapport

TABLE 2.1 – Cycle de vie du projet Lumière Cinémas

Chaque phase produit des livrables concrets qui alimentent la phase suivante. La phase de développement est découpée en itérations correspondant aux modules fonctionnels (Chapitre 6.10.5), ce qui permet de valider chaque fonctionnalité indépendamment avant d'intégrer la suivante.

2.2 Méthodologie de Développement

Le projet Lumière Cinémas a été réalisé selon une approche **itérative et incrémentale**, inspirée des principes agiles. Cette méthodologie a permis d'identifier rapidement les risques techniques, de valider les choix architecturaux, et de livrer des fonctionnalités opérationnelles à chaque itération.

Le développement s'est articulé autour de cinq grandes phases successives :

0. **Analyse** : Recueil des besoins fonctionnels et non-fonctionnels, étude des technologies, définition des acteurs et des cas d'usage.
0. **Conception** : Modélisation UML (cas d'usage, classes, séquences, activité), architecture Next.js, conception du schéma de données Supabase.
0. **Développement** : Implémentation incrémentale authentification, vitrine publique, tunnel de réservation, intégration Stripe, dashboard administrateur.
0. **Tests & Validation** : Vérification fonctionnelle de chaque module, tests de sécurité (protection des clés API, contrôle d'accès admin, cohérence des réservations concurrentes).
0. **Finalisation** : Revue du code, rédaction du rapport technique, préparation de la soutenance.

2.3 Découpage en Phases et Jalons

Le tableau suivant détaille le découpage chronologique du projet sur la période du **1^{er} mai 2026 au 10 juin 2026** (six semaines), avec la répartition des responsabilités entre les trois membres de l'équipe : **Wassim Lazim**, **Marwan Saadouni** et **Kamal Chaafie**.

Phase	Tâches principales	Début	Fin	Responsable(s)
P1	Analyse des besoins, identification des acteurs, étude technique	01/05	07/05	Kamal Chaafie
P2	Conception architecture, modèle de données, diagrammes UML	04/05	12/05	Wassim Lazim, Marwan Saadouni
P3a	Configuration : Next.js, Supabase, Stripe, variables d'env.	06/05	09/05	Marwan Saadouni
P3b	Développement back-end Supabase : schéma SQL, RLS, Auth	11/05	21/05	Wassim Lazim
P3c	Développement vitrine publique : accueil, carousel, séances	14/05	23/05	Marwan Saadouni
P3d	Tunnel de réservation : SeatPicker, snacks, draft cookie	20/05	01/06	Wassim Lazim

Phase	Tâches principales	Début	Fin	Responsable(s)
P3e	Intégration Stripe : Checkout, web-hook, confirmation	25/05	02/06	Wassim Lazim
P3f	Dashboard administrateur : CRUD films/séances/snacks, KPIs	27/05	06/06	Marwan Saadouni
P4	Tests fonctionnels manuels, correction des anomalies	03/06	09/06	Kamal Chaafie
P5	Rédaction du rapport technique, mise en forme	22/05	10/06	Kamal Chaafie
P6	Finalisation, revue de code, préparation soutenance	08/06	10/06	Tous les membres

2.4 Diagramme de Gantt

La figure 2.1 illustre la planification temporelle du projet du 1^{er} mai au 10 juin 2026. Les phases de développement se chevauchent partiellement selon une logique incrémentale : la rédaction du rapport débute à mi-parcours, en parallèle des développements avancés.

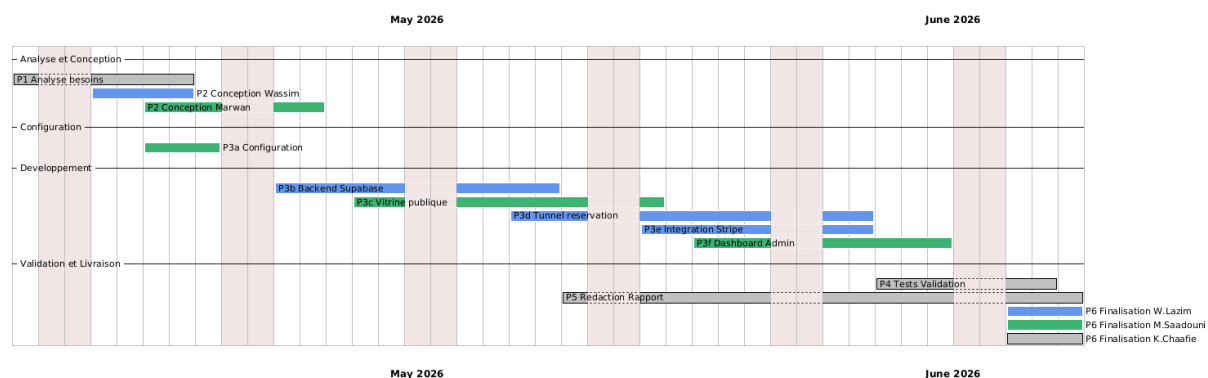


FIGURE 2.1 – Diagramme de Gantt Planification du projet Lumière Cinémas (01/05 10/06/2026)

Chapitre 3

Analyse des Besoins

3.1 Acteurs du Système

- **Visiteur (non authentifié)** : Consulte le catalogue de films, les fiches détaillées et les séances disponibles. Peut initier une réservation mais devra s'identifier avant le paiement.
- **Utilisateur (authentifié)** : Dispose de tous les droits du visiteur, plus la possibilité de réserver, payer, et consulter son historique de réservations dans son espace personnel.
- **Administrateur** : Accède au dashboard protégé pour gérer l'intégralité du contenu (films, séances, snacks) et consulter toutes les réservations et utilisateurs.

3.2 Besoins Fonctionnels

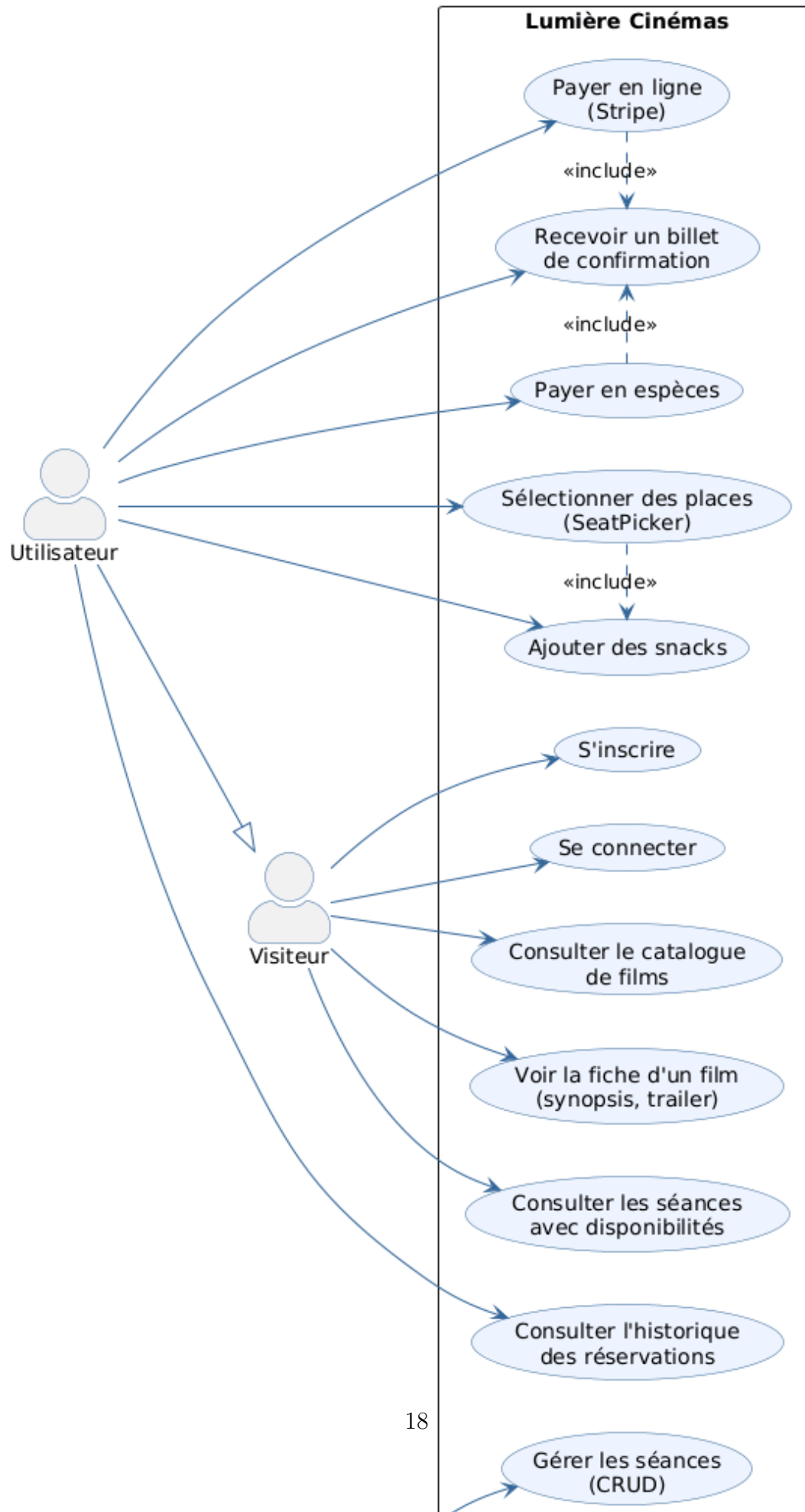
ID	Besoin	Acteur
BF-01	Consulter le catalogue de films (carousel + grille)	Visiteur
BF-02	Voir la fiche d'un film (synopsis, trailer, séances)	Visiteur
BF-03	Consulter les séances avec places restantes	Visiteur
BF-04	S'inscrire et se connecter (Supabase Auth)	Utilisateur
BF-05	Sélectionner des places sur plan interactif	Utilisateur
BF-06	Ajouter des snacks au panier	Utilisateur
BF-07	Payer via Stripe Checkout (carte bancaire)	Utilisateur
BF-08	Payer en espèces (réservation en attente)	Utilisateur
BF-09	Recevoir un billet de confirmation avec référence	Utilisateur
BF-10	Consulter l'historique de ses réservations	Utilisateur
BF-11	Gérer les films (CRUD)	Administrateur
BF-12	Gérer les séances (CRUD)	Administrateur
BF-13	Gérer les snacks (CRUD)	Administrateur
BF-14	Consulter toutes les réservations	Administrateur
BF-15	Consulter et gérer les utilisateurs	Administrateur
BF-16	Voir les statistiques (CA, films, séances, réservations)	Administrateur

3.3 Besoins Non-Fonctionnels

- **Sécurité** : Toutes les opérations sur la base de données transitent via des Server Actions côté serveur. Les clés Supabase service-role et Stripe secret ne sont jamais exposées au navigateur.
- **Performance** : Rendu SSR pour un premier affichage optimisé (First Contentful Paint) et un meilleur référencement SEO. Données chargées côté serveur au moment du rendu.
- **Cohérence des données** : Le plan de salle charge les sièges occupés depuis Supabase avant affichage. Les réservations confirmées marquent les sièges en temps réel.
- **Disponibilité** : Déploiement sur Vercel avec edge network global pour une latence minimale.
- **Maintenabilité** : TypeScript strict, composants réutilisables, Server Actions isolées par domaine fonctionnel.

3.4 Cas d'Usage

La figure [3.1](#) illustre les cas d'usage principaux du système.

Diagramme des Cas d'Usage — Lumière Cinémas

Chapitre 4

Architecture et Conception

4.1 Architecture Générale

L'application repose sur le paradigme **Server-Side Rendering (SSR)** et les **Server Actions** de Next.js 15. Cette architecture à deux niveaux (serveur / client) garantit que la logique métier critique et les clés d'API ne transitent jamais par le navigateur.

Flux de données global

Navigateur → **Next.js App Router** (React Server Components) → **Server Actions** (`actions.ts`) → **Supabase** (PostgreSQL) / **Stripe API**

La figure 4.1 présente l'architecture globale du système.

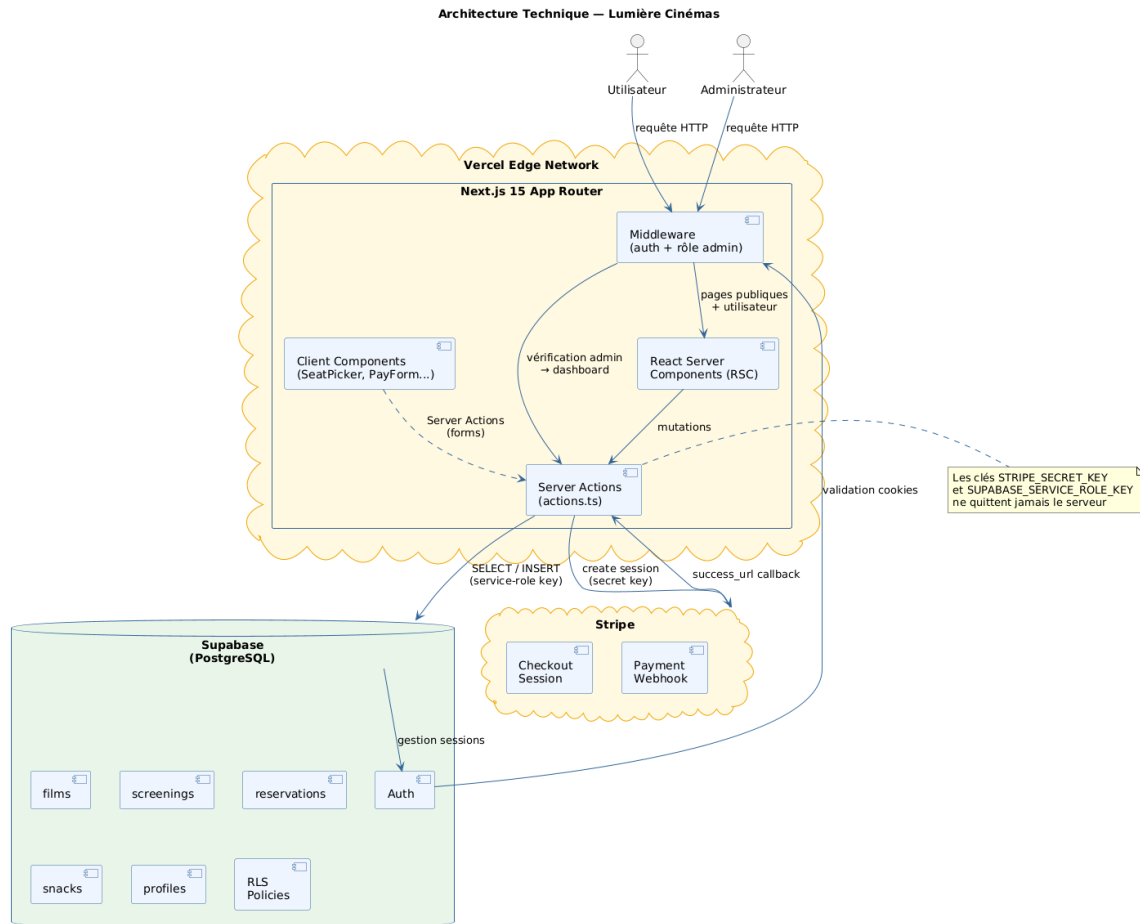


FIGURE 4.1 – Architecture technique Lumière Cinémas

4.2 Structure des Routes Next.js

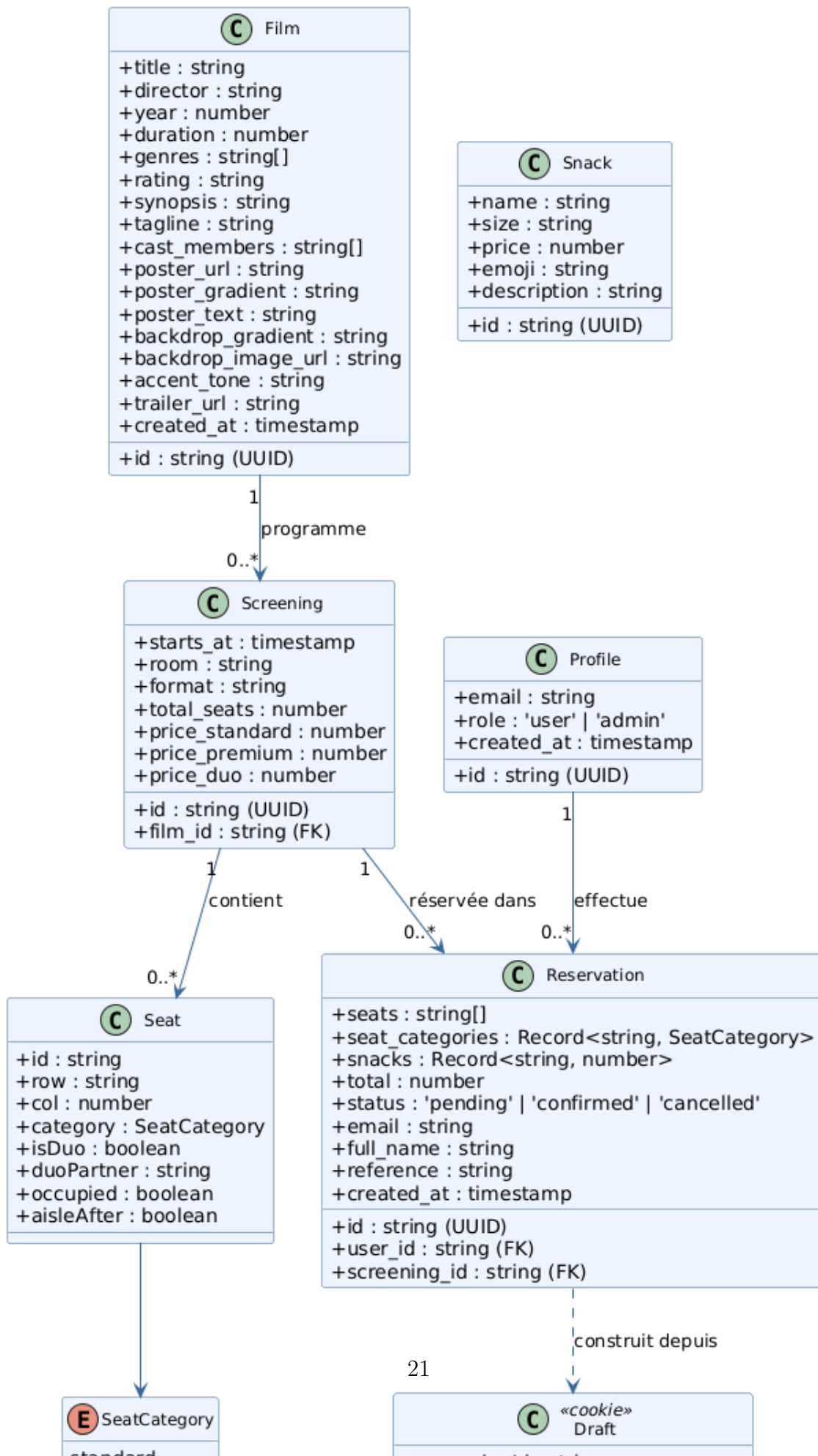
L'App Router de Next.js organise les pages selon la hiérarchie du système de fichiers :

- **Routes publiques :** /, /films/[id], /seances/[id], /programme, /prochainement, /salles
- **Routes authentification :** /login, /signup
- **Routes tunnel réservation :** /seances/[id], /snacks, /pay, /pay/success, /confirm/[id]
- **Routes utilisateur :** /account, /mes-reservations
- **Routes admin :** /dashboard, /dashboard/films, /dashboard/seances, /dashboard/snacks, /dashboard/reservations, /dashboard/utilisateurs

4.3 Modèle de Données

Le modèle logique de données est implémenté en PostgreSQL via Supabase. La figure 4.2 présente le diagramme de classes complet.

Diagramme de Classes — Modèle de données Lumière Cinémas



Les entités principales sont :

Table	Description
films	Catalogue des films : titre, réalisateur, année, durée, genres (tableau), casting, synopsis, tagline, URLs des affiches/backdrop, URL bande-annonce, gradient CSS
screenings	Séances planifiées : film associé (film_id), date/heure (starts_at), salle (room), format (VOSTFR, IMAX), nombre de sièges, prix par catégorie
snacks	Inventaire des confiseries : nom, taille, prix, emoji, description
reservations	Réservations : utilisateur, séance, tableau des sièges choisis (seats), catégories par siège (seat_categories), snacks (dictionnaire quantités), total, statut, référence unique
profiles	Profils utilisateurs synchronisés avec auth.users : rôle (user/admin), email

4.4 Gestion de l'État Le Draft Cookie

Le tunnel de réservation traverse plusieurs pages sans rechargement. L'état intermédiaire (séance choisie, sièges sélectionnés, snacks) est persisté dans un cookie HTTP via le module `lib/booking.ts` :

```

1 // lib/booking.ts
2 'use server';
3 export type Draft = {
4   screeningId: string;
5   seats: { id: string; category: 'standard' | 'premium' | 'duo'; price:
     number }[];
6   snacks: Record<string, number>;
7 };
8
9 export async function setDraft(d: Draft) {
10   const c = await cookies();
11   c.set('lumiere_draft', JSON.stringify(d), {
12     httpOnly: false, path: '/', maxAge: 60 * 60
13   });
14 }

```

Ce choix évite toute dépendance à un store global (Redux, Zustand) et reste compatible avec le SSR.

4.5 Diagramme d'Activité Tunnel de Réservation

La figure 4.3 illustre le flux d'activité complet d'une réservation.

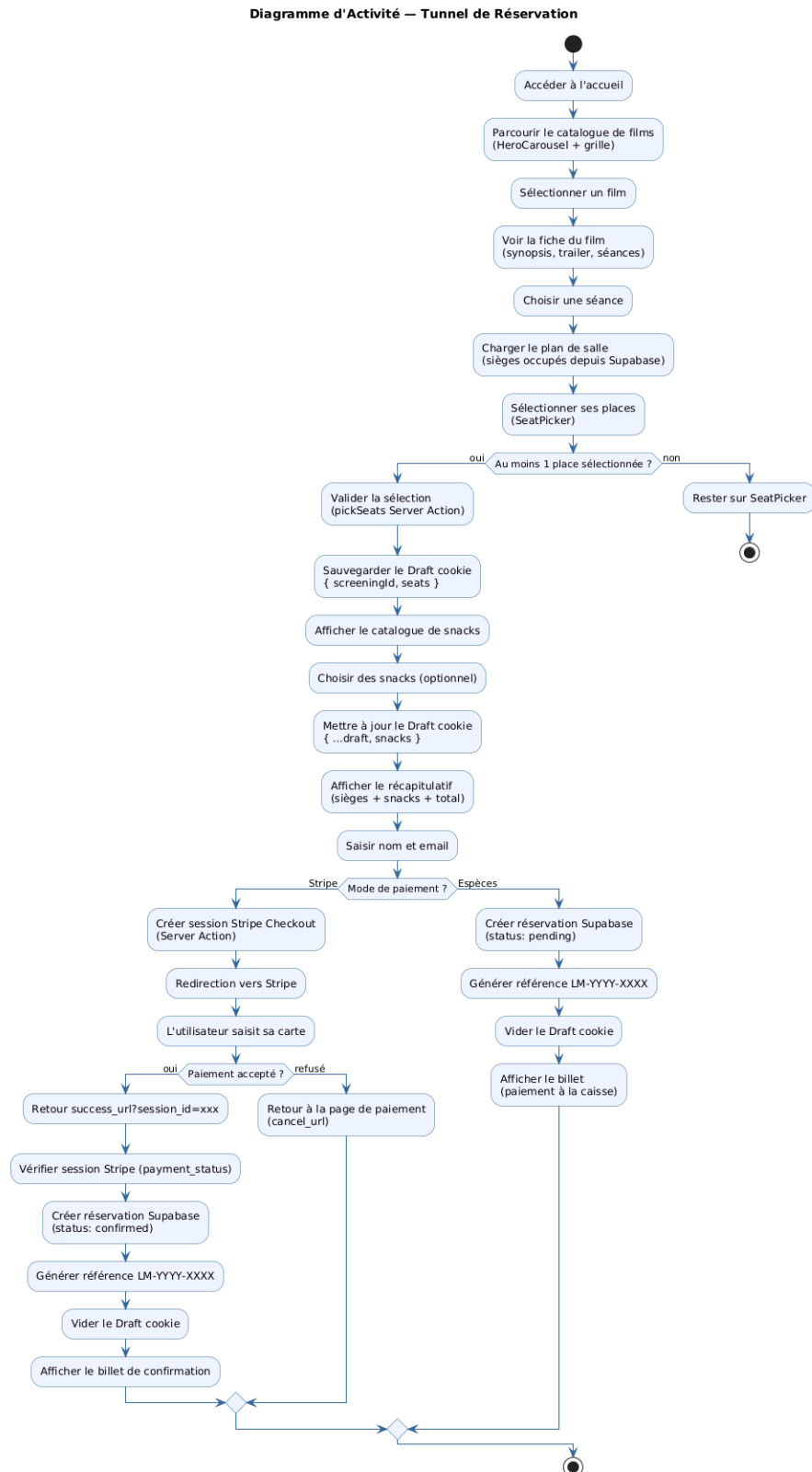


FIGURE 4.3 – Diagramme d'activité Tunnel de réservation

Chapitre 5

Stack Technologique

5.1 Tableau des Technologies

Catégorie	Technologie	Version
Framework Web	Next.js (App Router)	15.x
Langage	TypeScript	5.x
UI Library	React	19.x
Style	CSS personnalisé (variables)	
Base de données	Supabase (PostgreSQL)	Latest
Authentification	Supabase Auth (SSR)	@supabase/ssr
Paieement	Stripe	stripe 17.x
Composants UI	21st.dev	
Runtime	Node.js	18.x+
Minimum Node.js	18.x	

5.2 Justification des Choix

- **Next.js 15 App Router** : Architecture RSC native permettant le rendu serveur de chaque page, les Server Actions pour les mutations sécurisées, et le routage imbriqué par dossiers.
- **Supabase** : Alternative open-source à Firebase, fournissant une base PostgreSQL managée, un système d'authentification production-ready avec gestion des sessions par cookies (via `@supabase/ssr`), et des politiques RLS granulaires.
- **Stripe** : Standard de l'industrie pour le paiement en ligne. L'intégration via Stripe Checkout externalise la page de paiement, éliminant le besoin de manipuler des données de carte.
- **TypeScript strict** : Typage complet des entités (`Film`, `Screening`, `Reservation`, `Seat`) garantissant la cohérence entre la base de données et l'interface.
- **CSS personnalisé** : Variables CSS globales pour le thème visuel (couleurs, typographie) sans dépendance à un framework UI, permettant un contrôle pixel-perfect de l'interface.
- **21st.dev** : Bibliothèque de composants React/Next.js utilisée comme référence et source de composants UI prêts à l'emploi, accélérant le développement de l'interface

tout en maintenant une cohérence visuelle.

Chapitre 6

Fonctionnalités et Interface Utilisateur

6.1 Page d'Accueil Catalogue et HeroCarousel

La page d'accueil (`app/page.tsx`) est un React Server Component qui récupère la liste complète des films depuis Supabase au moment du rendu côté serveur. Elle présente deux sections principales :

0. **HeroCarousel** (`components/HeroCarousel.tsx`) : Un carrousel immersif affichant les films en vedette avec leurs affiches, gradients CSS dynamiques, et un appel à l'action « Voir les séances ».
0. **Grille de films** : Cartes cliquables affichant l'affiche, le titre, la durée, le genre et la classification de chaque film.

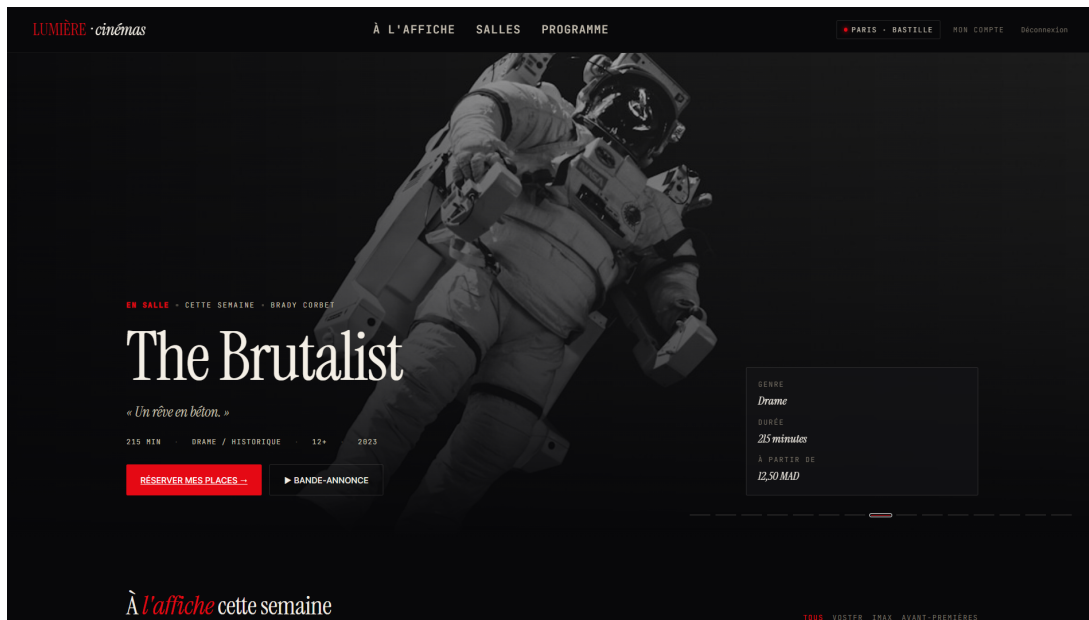


FIGURE 6.1 – Page d'accueil HeroCarousel et grille de films

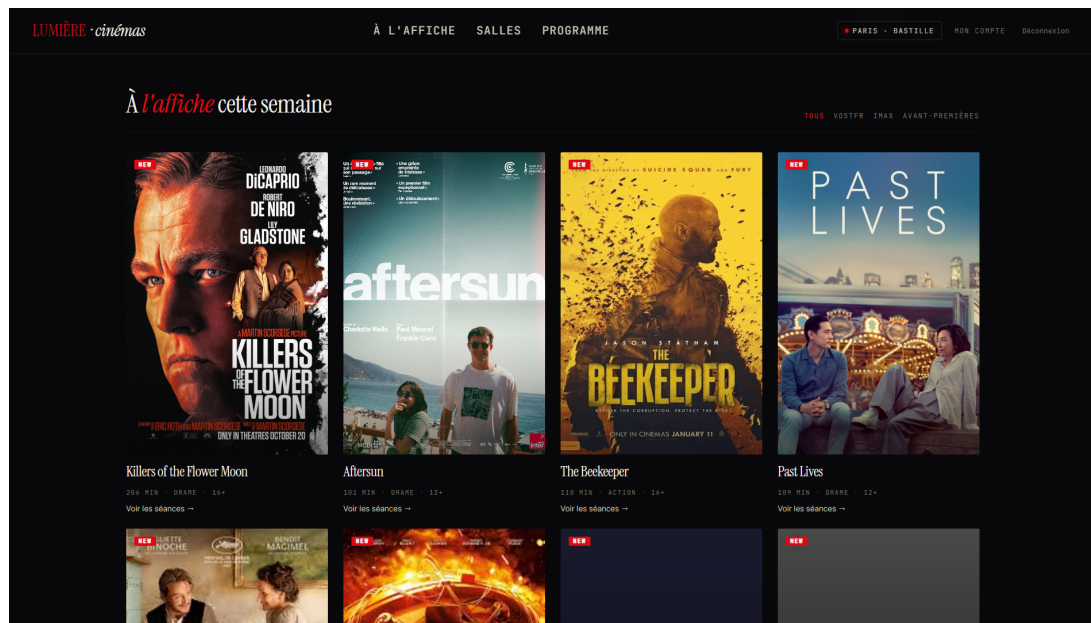


FIGURE 6.2 – Grille du catalogue de films

6.2 Fiche Film et Bande-Annonce

La page `app/films/[id]/page.tsx` affiche la fiche complète d'un film :

- En-tête hero avec backdrop gradient dynamique et affiche du film
- Métadonnées : réalisateur, durée, genres, classification
- Synopsis complet et listing du casting
- Bande-annonce YouTube intégrée (normalisation automatique de l'URL vers le format embed)
- Liste des séances disponibles avec places restantes, via `ScreeningsList`

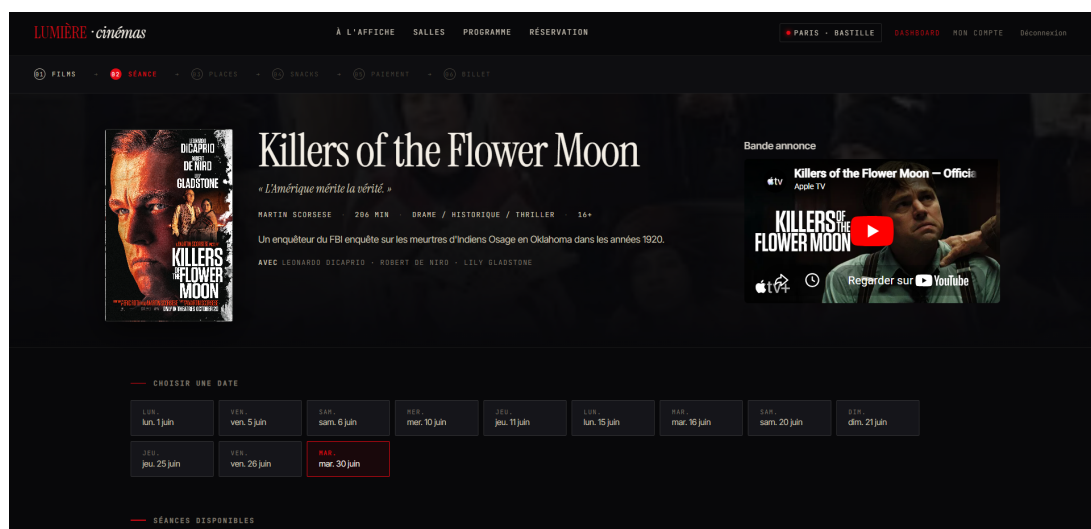


FIGURE 6.3 – Fiche détaillée d'un film avec bande-annonce et séances

6.3 Stepper Navigation Guidée

Le composant **Stepper** affiche la progression dans le tunnel de réservation. Il comporte 6 étapes définies statiquement :

01 Films → **02 Séance** → **03 Places** → **04 Snacks** → **05 Paiement** → **06 Billet**

Chaque page du tunnel reçoit la prop `current` pour indiquer l'étape active. Les étapes précédentes sont marquées `is-done`, l'étape courante `is-on`.

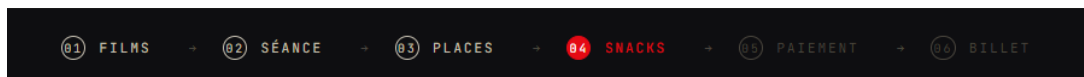


FIGURE 6.4 – Stepper indicateur de progression du tunnel

6.4 Sélection des Séances

La page `app/seances/[id]/page.tsx` charge côté serveur :

- 0. Les détails de la séance (film associé, salle, format, prix par catégorie)
- 0. Les sièges déjà réservés pour cette séance depuis la table `reservations`
- 0. La carte de la salle générée dynamiquement par `buildSeatMap()` depuis `lib/seatmap.ts`

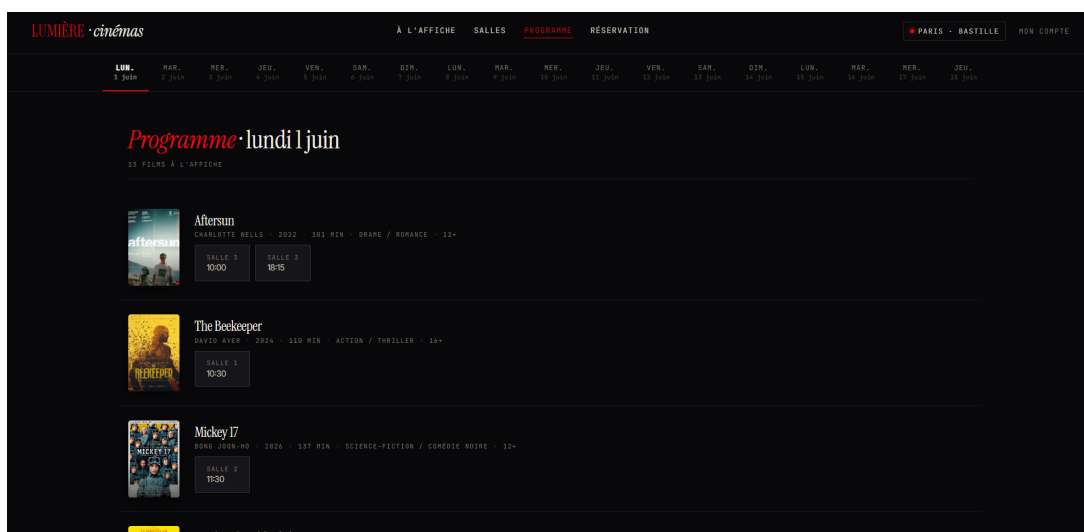


FIGURE 6.5 – Page de sélection de séance avec disponibilités

6.5 Sélection des Places SeatPicker

Le composant **SeatPicker** (`components/SeatPicker.tsx`) est le composant le plus complexe de l'application. Il est rendu côté client (`'use client'`) car il gère un état local interactif.

Caractéristiques :

- Affichage de la salle en 2D avec rangées (A à P) et colonnes numérotées

- **3 catégories de sièges** : Standard (gris foncé, 12,50 MAD), Premium (or, 16,00 MAD), Duo (rouge, 32,00 MAD)
- Sièges occupés désactivés visuellement (opacité réduite, non cliquables)
- Sièges sélectionnés mis en évidence avec la couleur d'accent
- Couloir de séparation central calculé automatiquement
- Panneau latéral récapitulatif avec liste des sièges sélectionnés, sous-total et bouton `Continuer`
- À la validation : appel de la Server Action `pickSeats()` qui sauvegarde le draft en cookie et redirige vers `/snacks`

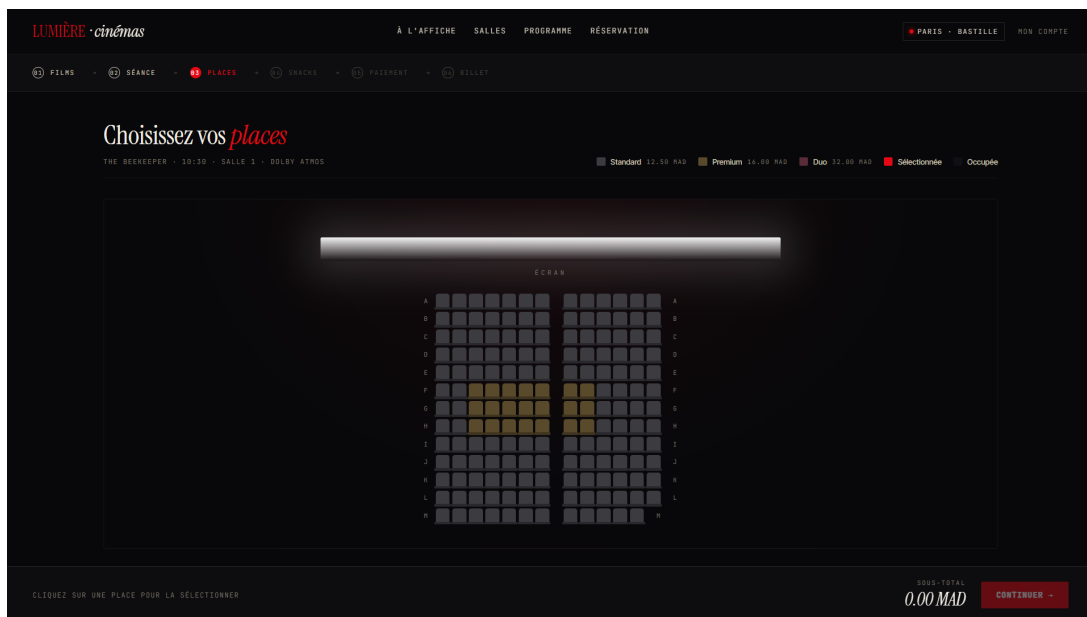


FIGURE 6.6 – Composant SeatPicker Plan interactif de la salle

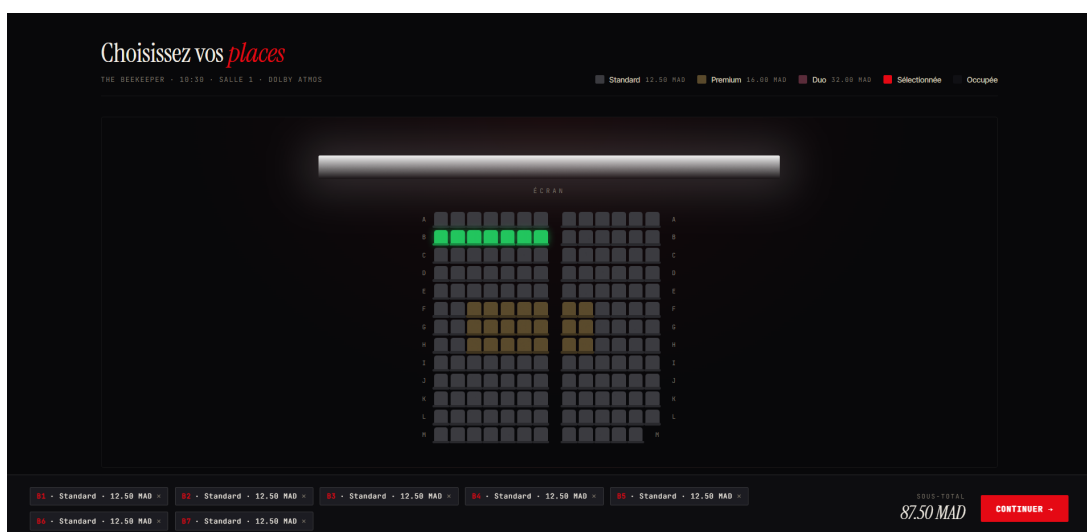


FIGURE 6.7 – SeatPicker avec sièges sélectionnés et panneau récapitulatif

6.6 Sélection des Snacks

La page `app/snacks/page.tsx` présente le catalogue de confiseries disponibles (récupéré depuis Supabase), permettant à l'utilisateur d'ajouter des articles à son panier avant de procéder au paiement. La Server Action `addSnacks()` met à jour le draft cookie avec les quantités choisies.

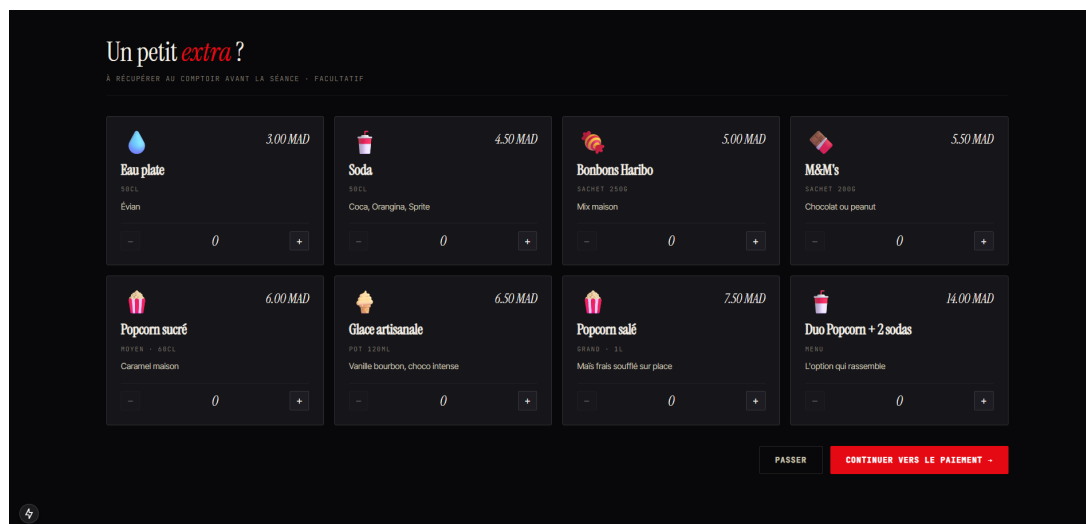


FIGURE 6.8 – Page de sélection des snacks

6.7 Paiement Stripe Checkout

La page `app/pay/page.tsx` affiche le récapitulatif final de la commande (sièges + snacks) et propose deux modes de règlement :

- **Paiement en ligne (Stripe)** : La Server Action `createCheckoutSession()` crée une session Stripe avec le montant calculé et redirige vers la page Stripe hébergée. Les métadonnées de réservation (siège, snacks, nom, séance) sont encodées dans la session Stripe pour être récupérées après paiement.
- **Paiement en espèces** : La Server Action `createCashReservation()` crée directement la réservation en base avec le statut `pending` et génère une référence unique (LM-YYYY-XXXX).

FIGURE 6.9 – Page de paiement récapitulatif et options de règlement

6.8 Confirmation et Billet

Après paiement Stripe validé, la page `/pay/success` est appelée avec le `session_id` Stripe. Elle :

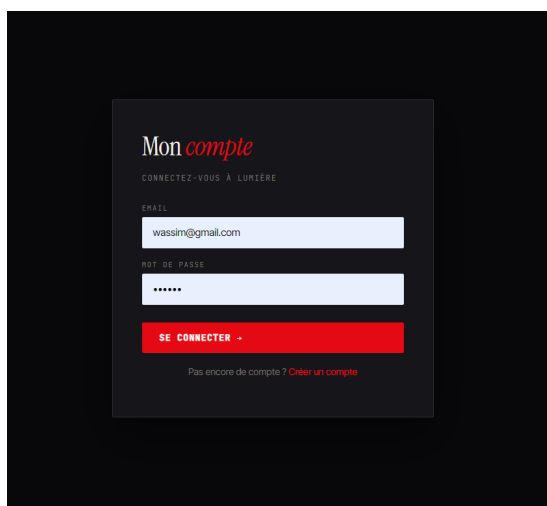
0. Vérifie le statut de la session Stripe (`payment_status === 'paid'`)
0. Relit les métadonnées encodées (sièges, snacks, nom)
0. Crée la réservation en base Supabase avec statut `confirmed`
0. Génère la référence de réservation (`LM-YYYY-XXXX`)
0. Redirige vers `/confirm/[id]` la page billet

FIGURE 6.10 – Billet de confirmation avec référence de réservation

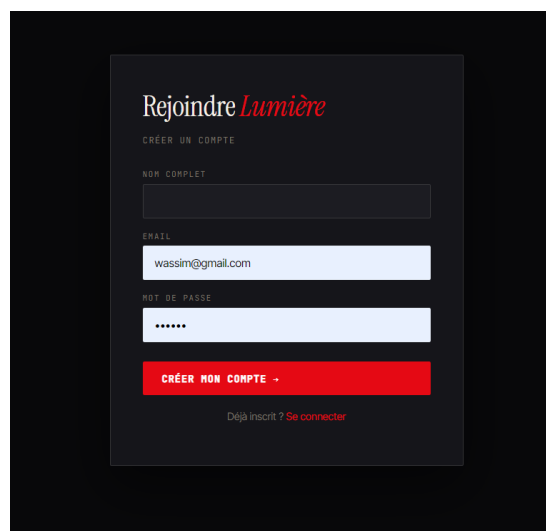
6.9 Espace Utilisateur

6.9.1 Authentification

L'inscription (`/signup`) et la connexion (`/login`) utilisent Supabase Auth via le package `@supabase/ssr`. Les sessions sont gérées par cookies HTTP-only, ce qui permet au middleware Next.js et aux Server Components de lire l'utilisateur courant sans appel côté client.



(a) Page de connexion



(b) Page d'inscription

FIGURE 6.11 – Pages d'authentification

6.9.2 Historique des Réservations

La page `/mes-reservations` affiche l'historique complet des billets achetés par l'utilisateur connecté, avec les détails de chaque séance et la liste des sièges réservés.

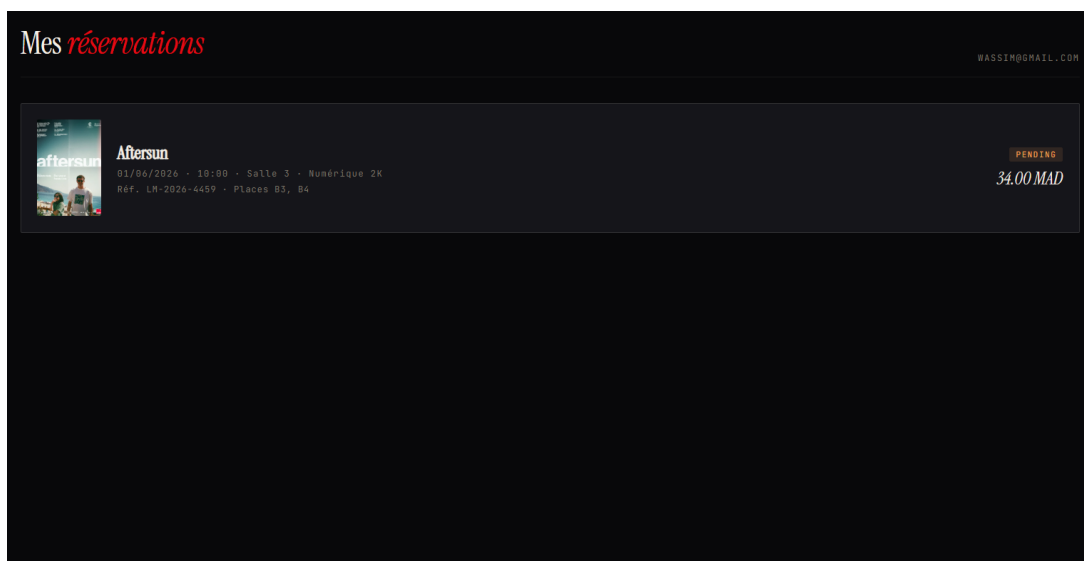


FIGURE 6.12 – Historique des réservations de l'utilisateur

6.10 Dashboard Administrateur

6.10.1 Protection et Accès

L'accès au dashboard est protégé par le layout `app/dashboard/layout.tsx` qui vérifie côté serveur :

0. L'existence d'une session active (Supabase Auth)
0. Le rôle `admin` dans la table `profiles`

Tout accès non autorisé est redirigé vers `/login` ou `/` selon le cas.

6.10.2 Vue d'Ensemble

La page `/dashboard` affiche 4 indicateurs clés calculés en temps réel depuis Supabase :

- Nombre total de films au catalogue
- Nombre total de séances programmées
- Nombre de réservations confirmées
- Chiffre d'affaires total des réservations confirmées (en MAD)

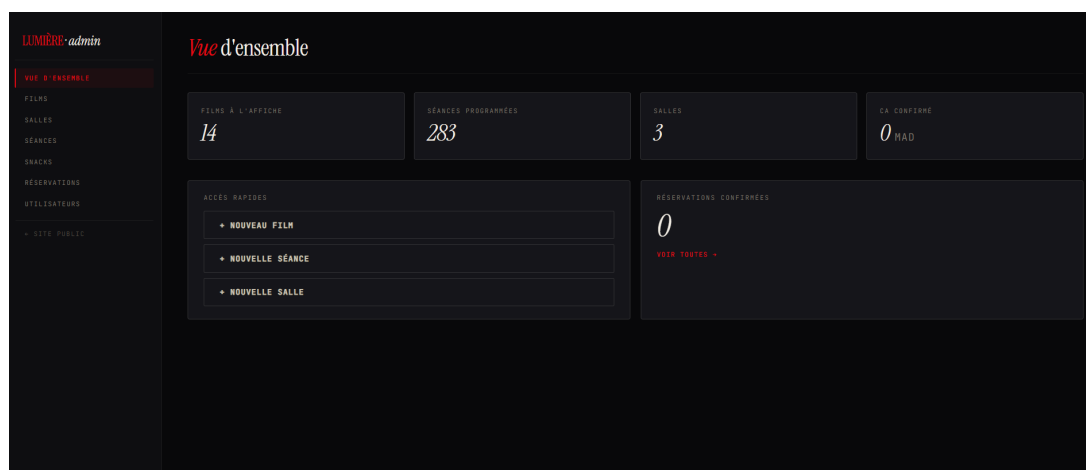


FIGURE 6.13 – Dashboard administrateur Vue d'ensemble et KPIs

6.10.3 Gestion des Films

La section `/dashboard/films` propose un CRUD complet : liste paginée, formulaire de création (`FilmForm.tsx`), formulaire d'édition, et suppression avec confirmation.

TITRE	REALISATEUR	ANNEE	DUREE	CLASSIF.	
Killers of the Flower Moon	Martin Scorsese	2023	206 min	16+	SUPPRIMER
The Bookkeeper	David Ayer	2024	110 min	16+	SUPPRIMER
Past Lives	Celine Song	2023	109 min	12+	SUPPRIMER
La Passion de Dodin Bouffant	Jacqueline Audry	1996	128 min	12+	SUPPRIMER
Oppenheimer	Christopher Nolan	2023	180 min	12+	SUPPRIMER
Dune: Part Two	Denis Villeneuve	2024	166 min	12+	SUPPRIMER
The Guiltiest	Brady Corbet	2023	215 min	12+	SUPPRIMER
The Iron Claw	Sean Durkin	2023	156 min	12+	SUPPRIMER
Anatomy of a Fall	Justine Triet	2023	149 min	16+	SUPPRIMER

FIGURE 6.14 – Dashboard Gestion des films (CRUD)

6.10.4 Gestion des Séances

La section /dashboard/seances permet de planifier de nouvelles séances (ScreeningForm.tsx) en associant un film, une date/heure, une salle, un format et des prix par catégorie de siège.

FILM	DATE & HEURE	SALLE	FORMAT	PLACES	
Altarus	01/06/2026 10:00:00	Salle 3 - Numérique 2K	VOSTFR - 2K	144	SUPPRIMER
The Bookkeeper	01/06/2026 10:30:00	Salle 1- Dolby Atmos	VOSTFR - 4K HDR	168	SUPPRIMER
Mickey 17	01/06/2026 11:30:00	Salle 2 - IMAX	VOSTFR - IMAX	220	SUPPRIMER
La Chambre d'Is Côté	01/06/2026 12:00:00	Salle 3 - Numérique 2K	VOSTFR - 2K	144	SUPPRIMER
The Iron Claw	01/06/2026 13:00:00	Salle 1- Dolby Atmos	VOSTFR - 4K HDR	168	SUPPRIMER
Past Lives	01/06/2026 14:00:00	Salle 3 - Numérique 2K	VOSTFR - 2K	144	SUPPRIMER
Megalopolis	01/06/2026 14:00:00	Salle 2 - IMAX	VOSTFR - IMAX	220	SUPPRIMER
Anatomy of a Fall	01/06/2026 15:30:00	Salle 1- Dolby Atmos	VOSTFR - 4K HDR	168	SUPPRIMER
La Passion de Dodin Bouffant	01/06/2026 16:00:00	Salle 3 - Numérique 2K	VOSTFR - 2K	144	SUPPRIMER
Dune: Part Two	01/06/2026 16:30:00	Salle 2 - IMAX	VOSTFR - IMAX	220	SUPPRIMER

FIGURE 6.15 – Dashboard Gestion des séances

6.10.5 Gestion des Réservations et Utilisateurs

RESERVATION	UTILISATEUR	FILM	SALLE	FORMAT	DATE
1234567890	user@lumiere.com	Altarus	Salle 3 - Numérique 2K	VOSTFR - 2K	01/06/2026

(a) Liste des réservations

EMAIL	NOM	ROLE	STATUT	ACTIONS
admin@lumiere.com	Admin	ADMIN	ACTIF	MODIFIER SUPPRIMER
user@lumiere.com	User	UTILISATEUR	ACTIF	MODIFIER SUPPRIMER

(b) Gestion des utilisateurs

FIGURE 6.16 – Dashboard Réservations et utilisateurs

Chapitre 7

Implémentation Technique

7.1 Génération du Plan de Salle

La fonction `buildSeatMap()` dans `lib/seatmap.ts` génère dynamiquement le plan de salle en fonction du nombre total de sièges de la séance :

```
1 export function buildSeatMap(  
2   occupied: Set<string> = new Set(),  
3   totalSeats: number = 112  
4 ): { row: string; seats: Seat[] }[] {  
5   const rowLabels = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',  
6     'N', 'O', 'P'];  
7   const cols = Math.max(6, Math.ceil(Math.sqrt(totalSeats)));  
8   const rows = Math.ceil(totalSeats / cols);  
9  
10  return rowLabels.slice(0, rows).map((r, ri) => {  
11    const seats: Seat[] = [];  
12    for (let c = 1; c <= cols; c++) {  
13      let category: SeatCategory = 'standard';  
14      const midCols = Math.floor(cols / 2);  
15      // Centre de la salle = Premium  
16      if (ri >= Math.floor(rows/2 - 1) && ri <= Math.floor(rows/2 + 1)  
17        && c >= midCols - 3 && c <= midCols + 3) {  
18        category = 'premium';  
19      }  
20      seats.push({  
21        id: `${r}${c}`, row: r, col: c, category,  
22        isDuo: false, duoPartner: null,  
23        occupied: occupied.has(`${r}${c}`),  
24        aisleAfter: c === Math.ceil(cols / 2),  
25      });  
26    }  
27    return { row: r, seats };  
28  });  
29 }
```

7.2 Création de la Session Stripe

La Server Action `createCheckoutSession()` dans `app/pay/actions.ts` assemble le montant total, encode les métadonnées de réservation et crée la session Stripe :

```

1  'use server';
2  export async function createCheckoutSession(formData: FormData) {
3    const draft = await getDraft();
4    if (!draft) redirect('/');
5
6    const supabase = await createClient();
7    const { data: snacks } = await supabase.from('snacks').select('id,
8      price');
9    const priceById = new Map((snacks || []).map(s => [s.id, Number(s.
10      price)]));
11
12    const seatsTotal = draft.seats.reduce((a, s) => a + s.price, 0);
13    const snacksTotal = Object.entries(draft.snacks).reduce(
14      (a, [id, q]) => a + (priceById.get(id) || 0) * q, 0
15    );
16
17    const session = await stripe.checkout.sessions.create({
18      payment_method_types: ['card'],
19      line_items: [{
20        price_data: {
21          currency: 'mad',
22          product_data: { name: 'Réservation cinéma' },
23          unit_amount: Math.round((seatsTotal + snacksTotal) * 100),
24        },
25        quantity: 1,
26      }],
27      mode: 'payment',
28      customer_email: email,
29      success_url: `${process.env.NEXT_PUBLIC_APP_URL}/pay/success?
30        session_id={CHECKOUT_SESSION_ID}`,
31      cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/pay`,
32      metadata: {
33        fullName, screeningId: draft.screeningId,
34        seats: JSON.stringify(draft.seats),
35        snacks: JSON.stringify(draft.snacks),
36      },
37    });
38
39    redirect(session.url!);
40  }

```

7.3 Protection du Dashboard Admin

Le layout serveur `app/dashboard/layout.tsx` vérifie l'authentification et le rôle avant de rendre les pages enfants :

```

1  export default async function DashboardLayout({ children }) {
2    const supabase = await createClient();
3    const { data: { user } } = await supabase.auth.getUser();
4    if (!user) redirect('/login?next=/dashboard');
5
6    const { data: profile } = await supabase
7      .from('profiles').select('role,email').eq('id', user.id).single();
8    if (profile?.role !== 'admin') redirect('/');
9
10   return <div className="dash">...{children}</div>;

```

11

}

Sécurité critique

Aucune clé d'API Supabase service-role ni clé Stripe secrète n'est incluse dans le bundle JavaScript envoyé au navigateur. Toutes les opérations sensibles transitent exclusivement par des Server Actions ou des Server Components.

7.4 Diagrammes de Séquence

7.4.1 Flux de Réservation Complet

La figure [7.1](#) illustre le diagramme de séquence du tunnel de réservation de bout en bout.

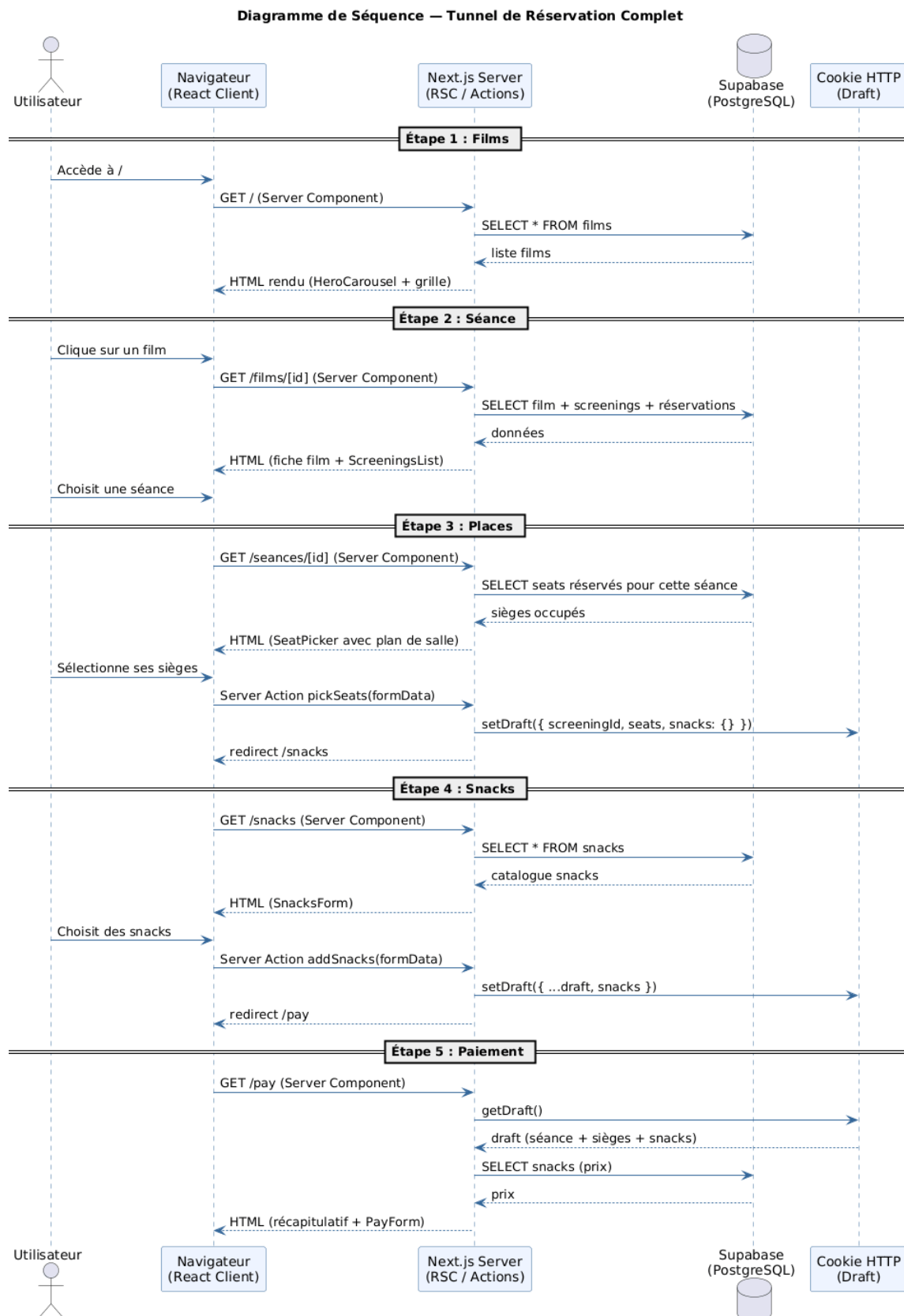


FIGURE 7.1 – Diagramme de séquence Tunnel de réservation

7.4.2 Flux de Paiement Stripe

La figure 7.2 illustre les interactions avec l'API Stripe lors du paiement en ligne.

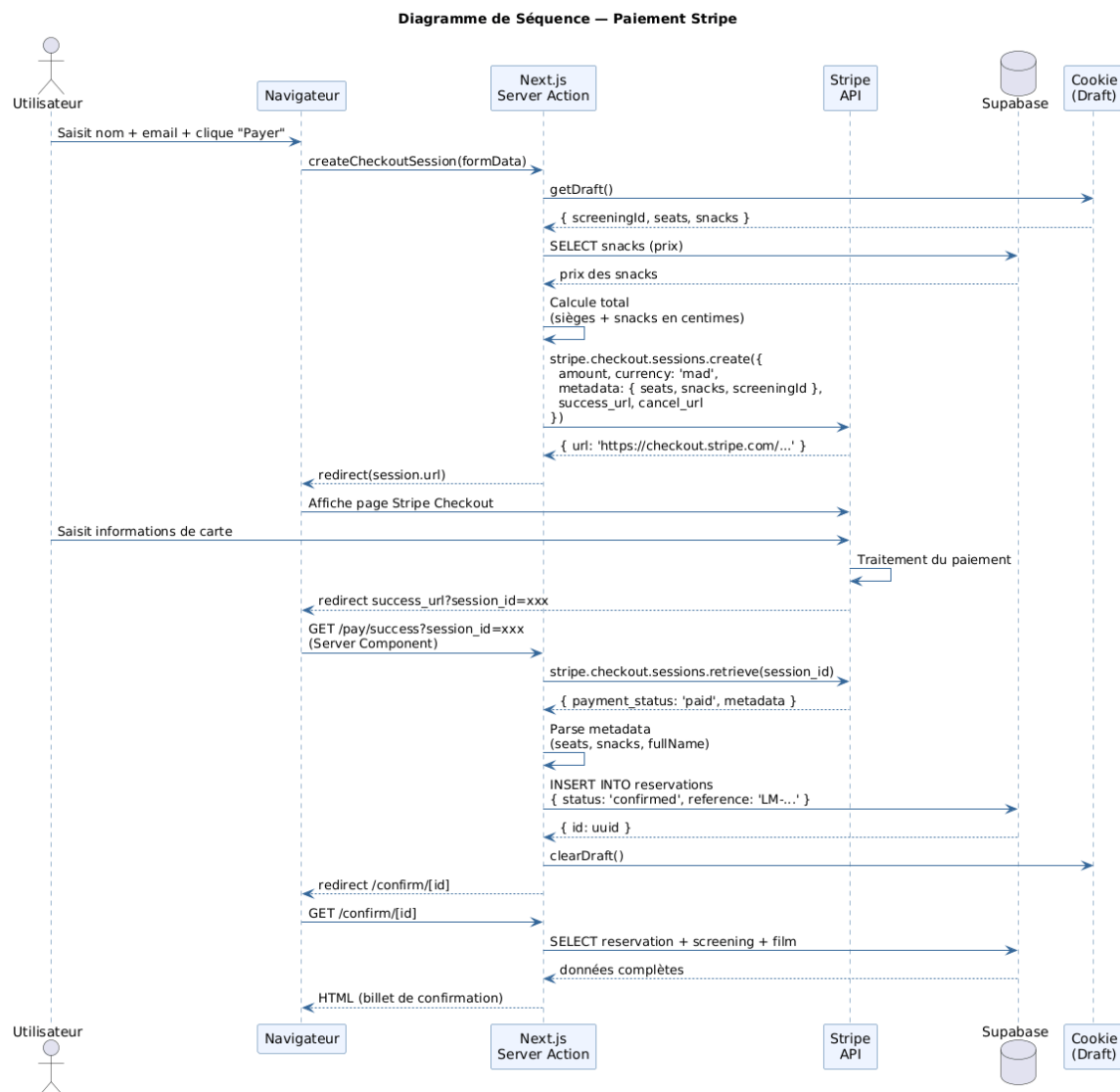


FIGURE 7.2 – Diagramme de séquence Paiement Stripe

Chapitre 8

Tests et Validation

8.1 Tests Fonctionnels Réalisés

Test	Description	Résultat
TF-01	Parcours complet de réservation (Stripe) : sélection séance → siège → snacks → paiement → confirmation	OK
TF-02	Parcours complet de réservation (espèces)	OK
TF-03	Blocage accès dashboard pour utilisateur non admin	OK
TF-04	Blocage accès dashboard sans authentification	OK
TF-05	Sièges occupés non sélectionnables dans SeatPicker	OK
TF-06	CRUD films depuis dashboard	OK
TF-07	CRUD séances depuis dashboard	OK
TF-08	CRUD snacks depuis dashboard	OK
TF-09	Affichage bande-annonce YouTube (normalisation URL)	OK
TF-10	Draft cookie expiré redirige vers accueil	OK
TF-11	Statistiques dashboard correctes (CA, comptages)	OK

8.2 Tests à Implémenter

- **Tests unitaires** (Jest) : Fonction `buildSeatMap()`, calcul du total (sièges + snacks), génération de référence
- **Tests d'intégration** (Jest + Supabase test DB) : Server Actions CRUD, politiques RLS
- **Tests End-to-End** (Playwright) : Tunnel de réservation complet, flux de connexion, dashboard admin

Chapitre 9

Mise en Route et Configuration

9.1 Variables d’Environnement

Le fichier `.env.local` à la racine du projet doit contenir les variables suivantes :

```
1 # Supabase
2 NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
3 NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
4 SUPABASE_SERVICE_ROLE_KEY=eyJ... # jamais exposee au client
5
6 # Stripe
7 STRIPE_SECRET_KEY=sk_test... # jamais exposee au client
8 NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test...
9
10 # App
11 NEXT_PUBLIC_APP_URL=http://localhost:3000
```

9.2 Lancement en Développement

```
1 npm install # installation des dependances
2 npm run dev # serveur de developpement (localhost:3000)
3 npm run build # verification du build de production
4 npm run lint # verification TypeScript + ESLint
```

L’application est accessible localement à l’adresse `http://localhost:3000`. Le serveur de développement Next.js assure le rechargement automatique à chaque modification du code source.

9.3 Base de Données Supabase

- Schéma initial créé via le fichier `supabase/schema.sql`
- Row-Level Security (RLS) activée sur toutes les tables
- Politique RLS `profiles` : les utilisateurs ne lisent que leur propre profil ; les admins lisent tout
- Politique RLS `reservations` : les utilisateurs ne voient que leurs propres réservations

- Trigger `handle_new_user()` : création automatique d'un profil dans `profiles` à chaque inscription

Chapitre 10

Perspectives Futures

10.1 Court Terme

- Implémenter tests automatisés (Playwright E2E, Jest unitaires)
- Notifications email de confirmation de réservation (Resend ou SendGrid)
- Optimiser les requêtes Supabase avec indexation sur `screening_id` et `user_id`
- Ajouter la pagination dans le dashboard (films, réservations)

10.2 Moyen Terme

- Intégration des sièges Duo réels (gestion des paires de sièges liés)
- Système de fidélité (points cumulés, réductions automatiques)
- Filtres avancés dans le catalogue (genre, format, date)
- Génération et envoi de QR-codes scannable à l'entrée de la salle
- Statistiques avancées dans le dashboard (taux d'occupation par salle, revenus par film)

10.3 Long Terme

- Extension multi-cinémas avec sélection de l'établissement
- Déploiement en production sur Vercel avec intégration GitHub (CI/CD automatisé)
- Application mobile compagnon (React Native / Expo)
- Système de prévente et avant-premières exclusives avec quota
- Intégration d'un système de recommandation de films personnalisé
- Monétisation avec places VIP et offres groupées (famille, scolaire)

Conclusion Générale

Le projet Lumière Cinémas a permis de concevoir et de développer une application web de réservation de cinéma complète, intégrant une vitrine publique, un tunnel de réservation guidé en 6 étapes, un système de paiement en ligne par carte bancaire, et un back-office administrateur sécurisé.

L'utilisation de Next.js 15 avec l'App Router et les Server Actions a permis de maintenir une séparation stricte entre le code serveur et le code client, garantissant la sécurité des clés d'API et la cohérence des données. L'intégration de Supabase comme backend-as-a-service et de Stripe pour le paiement a considérablement accéléré le développement tout en assurant un niveau de robustesse production.

Le composant **SeatPicker**, pièce maîtresse de l'application, démontre la capacité à concevoir des interfaces interactives complexes tout en maintenant la cohérence avec les données serveur en temps réel.

Ce projet constitue une base solide et extensible pour une application de billetterie professionnelle, prête à accueillir les évolutions décrites dans les perspectives futures.

Annexe A

Structure Complète du Projet

```
1  lumiere-cinemas/  
2  app/  
3      page.tsx                (accueil + HeroCarousel)  
4      layout.tsx              (layout global)  
5      films/[id]/page.tsx     (fiche film + seances)  
6      seances/[id]/  
7          page.tsx            (SeatPicker)  
8          actions.ts          (pickSeats Server Action)  
9      snacks/  
10         page.tsx  
11         actions.ts  
12     pay/  
13         page.tsx             (recapitulatif + paiement)  
14         actions.ts           (Stripe + cash Server Actions)  
15         success/page.tsx     (webhook Stripe)  
16     confirm/[id]/page.tsx    (billet de confirmation)  
17     mes-reservations/page.tsx  
18     account/page.tsx  
19     login/page.tsx  
20     signup/page.tsx  
21     programme/page.tsx  
22     prochainement/page.tsx  
23     salles/page.tsx  
24     auth/actions.ts          (login/signup/logout)  
25     auth/logout/route.ts  
26     api/films/[id]/route.ts  (API REST film)  
27     dashboard/  
28         layout.tsx           (protection admin)  
29         page.tsx             (vue d'ensemble + KPIs)  
30         films/               (CRUD films)  
31         seances/             (CRUD seances)  
32         snacks/              (CRUD snacks)  
33         reservations/page.tsx  
34         utilisateurs/  
35     components/  
36         HeroCarousel.tsx  
37         SeatPicker.tsx  
38         Stepper.tsx  
39         ScreeningsList.tsx  
40         ScreeningForm.tsx  
41         FilmForm.tsx  
42         SnacksForm.tsx  
43         SnackForm.tsx
```

```
44     PayForm.tsx
45     DeleteButton.tsx
46     Header.tsx
47     PosterBg.tsx
48   lib/
49     types.ts                (Film, Screening, Snack, Seat,
                             Reservation)
50     booking.ts              (getDraft / setDraft / clearDraft)
51     seatmap.ts              (buildSeatMap, formatDate, formatTime)
52     supabase/
53       client.ts             (client-side Supabase)
54       server.ts             (server-side Supabase SSR)
55   supabase/
56     schema.sql              (schema + triggers + RLS)
57   .env.local
58   next.config.ts
59   tsconfig.json
60   package.json
```

Annexe B

Ressources et Références

- 0. Next.js 15 Documentation : <https://nextjs.org/docs>
- 0. React 19 Documentation : <https://react.dev>
- 0. Supabase Documentation : <https://supabase.com/docs>
- 0. Supabase Auth avec Next.js SSR : <https://supabase.com/docs/guides/auth/server-side/nextjs>
- 0. Stripe Checkout Documentation : <https://stripe.com/docs/payments/checkout>
- 0. TypeScript Documentation : <https://www.typescriptlang.org/docs>
- 0. 21st.dev Bibliothèque de composants UI React/Next.js : <https://21st.dev>
- 0. Anthropic, *Claude* (modèle Sonnet 4.6) Assistant IA utilisé comme aide au développement et à la rédaction technique, <https://claude.ai>, .
- 0. Google DeepMind, *Gemini* Assistant IA utilisé comme aide à la recherche et à la résolution de problèmes techniques, <https://gemini.google.com>, .